TECHNICAL & PRODUCT REFERENCE

Formulate

A deep-learning pipeline that reads a printed math equation from an image and returns editable LaTeX and MathML — end to end, from the upload form to the trained model to the API response.

Flask + PyTorch | 34.4M parameters | 544-token vocabulary | im2latex-100k | 28 tests, CI-gated

CONTENTS

00 Overview

Reading a printed equation is trivial for a person; turning it back into usable markup by hand is not. Formulate closes that gap.

Upload a scan or photo of a printed formula, and a custom encoder–decoder network — trained on the im2latex-100k dataset — predicts the LaTeX token sequence. That LaTeX is converted to MathML, and both are shown next to a rendered preview so the result can be checked at a glance before it's copied into a paper, a document, or a web page.

The project spans a trained PyTorch model, a Flask web application and JSON API around it, and a vanilla-JS frontend with no build step — deployable as a container to Cloud Run with an attached GPU, or run entirely on CPU.

01 How it works

Five stages, each with a single, testable responsibility:

`Image` → `Preprocess` → `LWDSC-SA Encoder` → `BiLSTM + Attention Decoder` → `LaTeX + MathML`

- **Preprocess** — the image is converted to grayscale, cropped to its content bounding box, sharpness/contrast-enhanced, Otsu-binarized, and resized to a fixed 160×1024 (H×W) tensor normalized to `[0, 1]` (`app/deep_learning/preprocessing.py`).
- **LWDSC-SA Encoder** — short for Lightweight Depthwise-Separable-Conv + Spatial-Attention. Three depthwise-separable convolution blocks (32→64→128 channels) keep the parameter count small; a spatial-attention layer weights the regions that matter; a row-wise BiLSTM turns the resulting 2D feature map into a left-to-right sequence.
- **Sequence model** — a second, 2-layer BiLSTM (hidden size 512) refines that sequence before decoding.
- **Decoder** — a single-layer LSTM with Luong (dot-product) attention generates one LaTeX token at a time, attending back to the relevant part of the encoded sequence for each symbol it writes. Run twice per request: once **greedy** (always the single highest-probability token), once via **beam search** (keeps the top 5 candidate sequences at each step) — both are returned side by side.
- **Postprocessing** — the winning token sequence is joined into a LaTeX string, then `latex2mathml` converts it to MathML (`app/deep_learning/postprocessing.py`).

02 Model architecture

Defined in `app/deep_learning/model.py` , loaded once at process startup and held in memory for the life of the server (`app/deep_learning/inference.py` , `app/__init__.py`) — never reloaded per request.

COMPONENT	SHAPE / DETAIL
Input	`1 × 160 × 1024 (grayscale, H×W)`
Encoder conv channels	`32 → 64 → 128`
Row BiLSTM hidden size	`64 (×2, bidirectional)`
Sequence-model BiLSTM	`2 layers, 512 hidden, dropout 0.3`
Decoder embedding dim	`384`
Decoder hidden size	`512`
Vocabulary	`544 tokens (incl. <PAD> <SOS> <EOS> <UNK>)`
Total parameters	`34,437,067`
Beam width / max length	`5 / 160 tokens`

Parameter count and checkpoint compatibility measured directly with `scripts/verify_checkpoint.py` against `models/full_checkpoint.pt`.

> **On the architecture choice:** this is a lightweight CNN + BiLSTM + LSTM-attention encoder–decoder — the same family as the original 2016-era image-to-markup systems, not a Transformer. It's a coherent, working design, not a mistake, but current Transformer-decoder / ViT-encoder approaches generally out-perform this architecture class on long or deeply nested formulas, where self-attention handles long-range structure better than a recurrent decoder.

---

03 Training & data

Trained on **im2latex-100k**, split into:

SPLIT	EXAMPLES
Train	`75,243`
Validation	`8,365`
Test	`10,279`

Training ran with early stopping (patience 7, tracked on validation loss) and a learning-rate reduction on plateau, checkpointing the best validation loss seen so far. The recorded tail of the run (epochs 35–50) shows validation loss easing from 1.687 to

1.639 as the learning rate stepped down from 1e-4 to 5e-5, with the checkpoint shipped in `models/full_checkpoint.pt` saved at epoch 50.

Raw training images/CSVs (`dataset/` , `sample/`) are not committed to the repository — only the trained checkpoint and vocabulary are. The notebooks that produced them (`training/notebooks/`) are kept for reference, though they contain hardcoded paths from the original training machine and aren't directly re-runnable as committed.

---

04 Performance

Measured on the held-out test set (~10,279 examples), comparing greedy decoding against beam search:

METHOD	EXACT MATCH	BLEU	WER	CRR
● Greedy	33.43%	86.00	9.96%	89.85%
● Beam search	30.90%	84.99	10.03%	89.64%

CRR = character recognition rate. Source: saved output cells in `training/notebooks/training_all_metrics.ipynb` .

Reading these numbers honestly

01 Exact-match under-counts real usability. LaTeX has many textually different but visually identical forms — `\dots` vs. `\ldots` , `\left( \right)` vs. plain parens, extra grouping braces. All count as full failures under exact-string match even when the rendered output is indistinguishable. The character-level numbers (CRR ~90%, WER ~10%) are a better read on day-to-day usefulness: most equations need at most minor touch-ups, not a rewrite.

02 The notebook also reports a "Token Accuracy" of 5.33% (greedy) / 5.11% (beam). That figure is almost certainly a bug in the notebook's own metric computation, not a real signal — it's logically inconsistent with every other number recorded: if a third of full sequences are exact matches, per-token accuracy across the dataset must be well above 33%, not below 6%. CRR/WER/BLEU/Exact-Match are the trustworthy figures here.

03 These numbers predate a beam-search fix made in this project's hardening pass (see Engineering notes). Beam search's ranking previously had no length normalization, which systematically penalizes longer — often more complete — sequences; that is the likely reason it under-performed greedy above despite exploring a wider search space. Spot-checked on the three bundled example images after the fix: beam search recovered a token greedy had dropped, with no regression on the two that already matched exactly. A full re-run of this table was not possible in this environment (the dataset is no longer present locally), so the table reflects the last real measurement, not the current expected behavior.

---

05 Frontend

Server-rendered Jinja templates and vanilla JavaScript — no build step, no framework. Two pages: **Recognize** (the tool) and **About** (what it is, how it works, privacy).

Upload & validate
Drag-and-drop or click-to-browse dropzone; file type and size validated client-side against the same config the server enforces, before any network round trip.

Crop & rotate
A canvas-based adjust tool with draggable resize handles and rotate left/right, for a skewed photo or an overly loose crop — before the image is ever submitted.

Dual decoding view
Beam search shown as the primary result; greedy decoding available in a collapsed secondary panel, with token-level diff highlighting between the two.

Rendered preview
MathJax typesets the predicted LaTeX inline, with a plain-text fallback if the CDN script fails to load.

Result history
The last 8 uploads stay one click away, stored only in the browser's `localStorage` — never sent anywhere, clearable at any time.

Feedback voting
A lightweight thumbs up/down per result, tallied client-side.

Loading & empty states
A skeleton placeholder during inference and a proper empty state before the first upload, instead of blank space.

Theme & responsiveness
A three-state light/dark toggle (explicit choice beats system preference either direction), a mobile nav that closes on outside click/Escape, and a PWA manifest.

06 Backend & API

A Flask application factory (`app/__init__.py`) wires everything together: the model is loaded once at startup, security headers and a scoped Content-Security-Policy are attached to every response, and 404/413/429/500 all branch into a JSON body for `/api/*` or a styled HTML page for browser navigation.

METHOD	ROUTE	PURPOSE
GET	`/` , `/recognize`	The Recognize page
GET	`/about`	The About page
POST	`/api/predict`	Upload an image, get LaTeX + MathML back (rate-limited)
GET	`/healthz`	Model-load status and device, for uptime checks
GET	`/examples/<file>`	Serves the quick-start sample images
GET	`/robots.txt`	Crawler policy

`app/services/inference_service.py` sits between the routes and the model: it validates the upload, runs preprocessing, calls the model for both decoding strategies, converts to MathML, and encodes the preprocessed preview image — keeping `routes.py` HTTP-only.

Hardening

- **Rate limiting** — per-IP, in-memory (matches the single-worker deployment), configurable via `RATE_LIMIT_PREDICT` (default 10/minute). Inference is CPU/GPU-bound and only a handful of worker threads are available; this keeps one client from starving everyone else.
- **Security headers** — `X-Content-Type-Options` , `X-Frame-Options` , `Referrer-Policy` , and a `Content-Security-Policy` scoped to exactly the origins the app loads (no wildcards), plus `ProxyFix` for the correct scheme/host behind Cloud Run's reverse proxy.
- **Startup shape validation** — a dummy forward pass at boot confirms the configured input dimensions match what the checkpoint expects, failing loudly at startup instead of on a user's first real request.

07 Testing & CI

28 tests across four files, all runnable on CPU:

- `test_app_routes.py` — HTTP wiring: every route, error-path status codes, security headers, and a genuine end-to-end `/api/predict` happy path with real inference, plus a test that exercises actual rate-limit enforcement.
- `test_model_shapes.py` — forward-pass tensor shapes through the encoder/sequence-model/decoder with the configured hyperparameters (random weights, not the trained checkpoint).
- `test_preprocessing.py` / `test_postprocessing.py` — tensor shape/dtype/normalization, blank/garbage-image error handling, MathML fixups, token joining.

GitHub Actions (`.github/workflows/ci.yml`) runs `ruff check` and the full `pytest` suite on every push and pull request, scoped to `app/` and `tests/` — deliberately excluding the training notebooks, which weren't held to the same lint bar. The model checkpoint is fetched via Git LFS and cached across runs, keyed on the LFS pointer hash, to stay inside GitHub's free LFS bandwidth quota.

---

08 Deployment

The `Dockerfile` builds on an official NVIDIA CUDA runtime image, so the same image runs GPU-accelerated on Cloud Run or falls back to CPU automatically wherever no driver is present (`DEVICE=auto`). It runs as a non-root user, with Gunicorn bound to a single worker and four threads — the model is loaded once per worker process, and GPU memory isn't safely shared across multiple workers.

The reference `cloudrun-service.yaml` attaches an NVIDIA L4 GPU with autoscaling from 0 to 3 instances. Recommended concurrency is kept low (4–8 per instance) since a single loaded model serializes inference work — this is a single-model-per-container design, not a multi-tenant batching server.

VARIABLE	DEFAULT
`DEVICE`	auto
`MAX_UPLOAD_MB`	8
`RATE_LIMIT_PREDICT`	10 per minute
`PORT`	8080
`MODEL_PATH` / `VOCAB_PATH`	`models/full_checkpoint.pt` / `models/vocab.json`

09 Limitations

- **The GPU path has never been exercised in this environment.** CUDA initialization, GPU tensor operations, and Cloud Run GPU deployment behavior are all unverified — every test and measurement in this document ran on CPU.
- **The Docker image hasn't been rebuilt since Flask-Limiter was added** as a runtime dependency; the recorded ~5.9GB image size and cold-start timing predate it.
- **Training notebooks aren't portable as committed** — they contain hardcoded absolute paths from the original training machine and would need editing to re-run.
- **Best on clear, high-contrast, printed (not handwritten) equations**, one equation per image rather than a full page of mixed text and math.
- **Not a multi-tenant batching server** by design — a handful of concurrent users is fine; high sustained concurrent load will queue rather than parallelize, since the model is single-instance per container.

10 Project structure

```
app/
  config.py                  Environment-driven configuration
  routes.py                  Flask routes (HTTP only)
  extensions.py              Shared extensions (Flask-Limiter)
  __init__.py                App factory; loads the model once at startup
  services/
    inference_service.py     Glue between routes and the DL pipeline
  deep_learning/
    model.py                 Network architecture
    preprocessing.py         Image → tensor pipeline
    inference.py              Model loading + greedy/beam decoding
    postprocessing.py        Token joining + LaTeX → MathML
  templates/, static/        Web UI
models/                      full_checkpoint.pt, vocab.json
training/                    Original training/preprocessing notebooks
examples/                    Quick-start sample images + ground truth
scripts/                     Local dev/verification helpers
tests/                       Pytest suite, 28 tests
.github/workflows/           CI: lint + test on push/PR
wsgi.py                      Production entrypoint / local dev runner
Dockerfile, cloudrun-service.yaml  Container build + deploy manifest
```

11 Engineering notes

A record of concrete bugs found and fixed, not just cosmetic polish — each verified directly, not assumed.

Correctness & model

✓ **Beam search had no length normalization**

Ranking summed raw log-probabilities, which systematically favors shorter sequences. The team's own recorded eval showed beam search under-performing greedy despite a wider search — exactly that symptom. Fixed by ranking on score ÷ length; verified beam search recovers a token greedy had dropped, on real ground-truth data.

✓ **Beam search wasn't batched**

One Python-loop decoder call per beam candidate, when the model already supported an arbitrary batch dimension. Refactored to one batched call per step; diffed the output against the original algorithm on real images first — bit-for-bit identical before changing anything.

✓ **No startup validation that config matched the checkpoint**

A wrong `IMG_HEIGHT` / `IMG_WIDTH` would previously boot fine and fail confusingly on the first real request. Added a dummy forward pass at startup; confirmed it fails loudly and clearly when deliberately mismatched.

Frontend

✓ **The crop "Adjust" button popped a native file dialog**

It sat inside the `<label>` wrapping the hidden file input without stopping propagation, so every click also bubbled up and triggered the OS file picker underneath the modal that had just opened.

✓ **The crop modal had no focus management or scroll lock**

Background page could still scroll behind an open modal; Tab could leak focus onto the page underneath. Added a focus trap, initial focus, focus restore on close, and a body scroll lock.

✓ **Mobile nav didn't close on outside click or Escape**

Only the toggle button or a link click closed it. Added both.

✓ **Hero copy didn't align with the page's own cards**

A leftover `max-width: 46rem` stopped the About page's intro text well short of the full-width cards below it. Widened to match the container's actual content width.

Repository hygiene

✓ **Missing test coverage for the actual happy path**

Every existing test hit an error branch of `/api/predict`; none confirmed a valid image returns 200 with the response shape the frontend depends on. Added it, plus a test that exercises real rate-limit enforcement rather than just checking the decorator is present.

✓ **Dead code and dead config**

An unused constructor parameter and an unused attribute in the decoder; five CSS classes and one CSS hook left over from a removed page; an `app.extensions` value written but never read. Found via cross-reference scripts, not eyeballing, then removed.

✓ **Legacy prototype, duplicate notebook, and orphaned diagnostics removed**

The original Streamlit prototype, a byte-for-byte duplicate training notebook (differed only in one developer's local file paths), and a folder of dataset-diagnostic output describing a dataset that's no longer present locally.

- ✓ **Full documentation staleness audit**

  The README still described a page that had been removed, referenced a "Next Steps" section that didn't exist anywhere in the file, and used the pre-rebrand image tag in its own deploy commands. Fixed across the README, `.env.example`, and the Cloud Run manifest.
